

Credit Risk Estimation

Although market risk is much better researched, the larger part of banks' economic capital is generally used for credit risk. The sophistication of traditional standard methods of measurement, analysis, and management of credit risk might, therefore, not be in line with its significance.

— Uwe Wehrspohn (2002)

The primary role of financial institutions is to create a channel by which funds move from entities with surplus into ones with deficit. Thereby, financial institutions ensure the capital allocation in the financial system as well as gain profit in exchange for these transactions.

However, there is an important risk for financial institutions to handle, which is credit risk. This is such a big risk that without it capital allocation might be less costly and more efficient. *Credit risk* is the risk that arises when a borrower is not able to honor their debt. In other words, when a borrower defaults, they fail to pay back their debt, which causes losses for financial institutions.

Credit risk and its goal can be defined in a more formal way (BCBS and BIS 2000):

Credit risk is most simply defined as the potential that a bank borrower or counterparty will fail to meet its obligations in accordance with agreed terms. The goal of credit risk management is to maximise a bank's risk-adjusted rate of return by maintaining credit risk exposure within acceptable parameters.

Estimating credit risk is so formidable a task that a regulatory body, Basel, closely monitors recent developments in the financial markets and sets regulations to strengthen bank capital requirements. The importance of having strong capital requirements for a bank rests on the idea that banks should have a capital buffer in turbulent times.

There is a consensus among policy makers that financial institutions should have a minimum capital requirement to ensure the stability of the financial system because a series of defaults may result in a collapse in financial markets, as financial institutions provide collateral to one another. Those looking for a workaround for this capital requirement learned their lessons the hard way during the **2007—2008 mortgage crisis**.

Of course, ensuring at least a minimum capital requirement is a burden for financial institutions in the sense that capital is an asset they cannot channel to deficit entities to make a profit. Consequently, managing credit risk amounts to profitable and efficient transactions.

In this respect, this chapter shows how credit risk can be estimated using cutting-edge ML models. We start our discussion with a theoretical background of credit risk. Needless to say, there are many topics in credit risk analysis, but we confine our focus on probability of default and how we can introduce ML approaches for estimating it. For this purpose, customers are segmented via a clustering method so that models can be separately fitted to this data. This provides a better fit in the sense that the distribution of credit risk data changes across different customer segments. Given the clusters obtained, Machine Learning model, is introduced to model the credit risk.

Estimating the Credit Risk

Aside from the probability of default (which is the likelihood that a borrower fails to cover their debt), credit risk has three defining characteristics:

Exposure

This refers to a party that may possibly default or suffer an adverse change in its ability to perform.

Likelihood

The likelihood that this party will default on its obligations.

Recovery rate

How much can be retrieved if a default takes place.

The BCBS put forth the global financial credit management standards, which are known as the *Basel Accord*. There are currently three Basel Accords. The most distinctive rule set by Basel I in 1988 was the requirement to hold capital equating to at least 8% of risk-weighted assets.

Basel I includes the very first capital measurement system, which was created following the onset of the **Latin American debt crisis**. In Basel I, assets are classified as follows:

- 0% for risk-free assets
- 20% for loans to other banks
- 50% for residential mortgages
- 100% for corporate debt

In 1999, Basel II issued a revision to Basel I based on three main pillars:

- Minimum capital requirements, which sought to develop and expand the standardized rules set out in the 1988 Accord
- Supervisory review of an institution's capital adequacy and internal assessment process
- Effective use of disclosure as a lever to strengthen market discipline and encourage sound banking practices

The last accord, Basel III in 2010, was inevitable. as the 2007–2008 mortgage crisis heightened. It introduced a new set of measures to further strengthened liquidity and poor governance practices. For instance, equity requirements were introduced to prevent a serial failure in the financial system, known as *domino effect*, during times of financial turbulence and crises. Accordingly, Basel III requires the financial ratios for banks listed in [Table 6-1](#).

Table 6-1. Financial ratios required by Basel III

Financial ratio	Formula
Tier 1 capital ratio	$\frac{\text{Equity capital}}{\text{Risk weighted assets}} > = 4.5 \%$
Leverage ratio	$\frac{\text{Tier 1 capital}}{\text{Average total assets}} > = 3 \%$
Liquidity coverage ratio	$\frac{\text{Stock of high quality liquid assets}}{\text{Total net cash outflows over the next 30 calendar days}} > = 100 \%$

Basel II suggests banks implement either a standardized approach or an internal ratings-based (IRB) approach to estimate the credit risk. The standardized approach is out of the scope of this book, but interested readers can refer to the “Standardized Approach to Credit Risk” [consultative document from the BIS](#).

Let's now focus on the IRB approach; the key parameters of this internal assessment are:

$$\text{Expected loss} = \text{EAD} \times \text{LGD} \times \text{PD}$$

where *PD* is the probability of default, *LGD* is the expected loss given default taking a value between 0 and 1, and *EAD* is the exposure at default.

The most important and challenging part of estimating credit risk is to model the probability of default, and the aim of this chapter is mainly to come up with an ML model to address this issue. Before moving forward, there is one more important issue in estimating credit risk that is sometimes neglected or overlooked: *risk bucketing*.

Risk Bucketing

Risk bucketing is nothing but grouping borrowers with similar creditworthiness. The behind-the-scenes story of risk bucketing is to obtain homogenous groups or clusters so that we can better estimate the credit risk. Treating different risky borrowers equally may result in poor predictions because the model cannot capture entirely different characteristics of the data at once. Thus, by dividing the borrowers into different groups based on riskiness, risk bucketing enables us to make accurate predictions.

Risk bucketing can be accomplished via different statistical methods, but we will apply a clustering technique to end up with homogeneous clusters using K-means.

We live in the age of data, but that does not necessarily mean that we always find the data we are searching for. Rather, it is rare to find it without applying data-wrangling and cleaning techniques.

Data with dependent variables is, of course, easy to work with and also helps us get more accurate results. However, sometimes we need to unveil the hidden characteristics of the data—that is, if the riskiness of the borrowers is not known, we are supposed to come up with a solution for grouping them based on their riskiness.

Clustering is the method proposed to create these groups or *clusters*. Optimal clustering has clusters located far away from one another spatially:

Clustering groups data instances into subsets in such a manner that similar instances are grouped together, while different instances belong to different groups. The instances are thereby organized into an efficient representation that characterizes the population being sampled.

— Rokach and Maimon (2005)

Different clustering methods are available, but the K-means algorithm serves our purpose, which is to create risk bucketing for credit risk analysis. In K-means, the distance of observations within the cluster is calculated based on the cluster center, the *centroid*. Depending on the distance to the centroid, observations are clustered. This distance can be measured via different methods. Of them, the following are the most well-known metrics:

Euclidean

$$\sqrt{\sum_{i=1}^n (p_i - q_i)^2}$$

Minkowski

$$(\sum_{i=1}^n |p_i - q_i|^p)^{1/p}$$

Manhattan

$$\sqrt{\sum_{i=1}^n |p_i - q_i|}$$

The aim in clustering is to minimize the distance between the centroid and observations so that similar observations will be on the same cluster. This logic rests on the intuition that the more similar observations are, the smaller the distance between them. So we are seeking to minimize the distance between observations and the centroid, which is another way of saying that we are minimizing the sum of the squared error between the centroid and the observations:

$$\sum_{i=1}^K \sum_{x \in C_i} (C_i - x)^2$$

where x is observation and C_i is the centroid of i^{th} cluster. However, considering the number of observations and the combinations of clusters, the search area might be too big to handle. It may sound intimidating, but don't worry: we have the *expectation-maximization (E-M)* algorithm behind our clustering. As K-means does not have a closed-form solution, we are searching for an approximate one, and E-M provides this solution. In the E-M algorithm, E refers to assigning observations to the nearest centroid, and M denotes completion of the data generation process by updating the parameters.

In the E-M algorithm, the distances between observations and the centroid is iteratively minimized. The algorithm works as follows:

1. Pick k random points to be centroids.
2. Based on the distance metric chosen, calculate the distances between observations and n centroids. Based on these distances, assign each observation to the closest cluster.
3. Update cluster centers based on the assignment.
4. Repeat the process from step 2 until the centroid does not change.

Now, we apply risk bucketing using K-means clustering. To decide the optimal number of clusters, different techniques will be employed. First, we use the *elbow method*, which is based on the *inertia*.

Inertia is computed as the sum of the squared distances of observations to their closest centroid. Second, the *Silhouette score* is introduced as a tool to decide the optimal number of clusters. This takes a value between 1 and -1. A value of 1 indicates that an observation is close to the correct centroid and correctly classified. However, -1 shows that an observation is not correctly clustered. The strength of the Silhouette score rests on taking into account both the intracluster distance and the intercluster distance. The formula for Silhouette score is as follows:

$$\text{Silhouette score} = \frac{x - y}{\max(x, y)}$$

where x is the average intercluster distance between clusters, and y is the mean intra-cluster distance.

The third method is *Calinski-Harabasz (CH)*, which is known as the *variance ratio criterion*. The formula for the CH method is as follows:

$$\text{CH} = \frac{SS_B}{SS_W} \times \frac{N - k}{k - 1}$$

where SS_B denotes between-cluster variance, SS_W represents within cluster variance, N is number of observations, and k is the number of clusters. Given this information, we are seeking a high CH score, as the larger (lower) the between-cluster variance (within cluster variance), the better it is for finding the optimal number of clusters.

The final approach is *gap analysis*. Tibshirani et al. (2001) came up with a unique idea by which we are able to find the optimal number of clusters based on reference distribution. Following the similar notations of Tibshirani et al., let d_{ii^e} be a Euclidean distance between x_{ij} and x_{i^ej} and let C_r be the i_{th} cluster denoting the number of observations in cluster r :

$$\sum_j \left(x_{ij} - x_{i^ej} \right)^2$$

The sum of pairwise distances for all observations in cluster r is:

$$D_r = \sum_{i, i^e \in C_r} d_{i, i^e}$$

The within-cluster sum of squares, W_k , is:

$$W_k = \sum_{r=1}^k \frac{1}{2n_r} D_r$$

where n is the sample size and expectation of W_k is:

$$W_k = \log(pn/12) - (2/p)\log(k) + \text{constant}$$

where p and k are dimension and centroids, respectively. Let's create a practice exercise using German credit risk data. The data is gathered from the [Kaggle platform](#), and the explanations of the variables are shown here:

- Age: Numerical
- Sex: Male, female
- Job: 0—unskilled and non-resident, 1—unskilled and resident, 2—skilled, 3—highly skilled
- Housing: Own, rent, free
- Saving accounts: Little, moderate, quite rich, rich
- Checking account: Numerical
- Credit amount: Numerical
- Duration: Numerical
- Purpose: Car, furniture/equipment, radio/TV, domestic appliances, repairs, education, business, vacation/others

The estimate of the optimal clusters will be the value that maximizes the gap statistic, as the gap statistic is the difference between the total within-intracluster variation for different values of k and their expected values under null reference distribution of the respective data. The decision is made when we get the highest gap value.

In the following code block, we import the German credit dataset and drop the unnecessary columns. The dataset includes both categorical and numerical values, which need to be treated differently, and we will do this soon:

```
In [1]: import pandas as pd

In [2]: credit = pd.read_csv('credit_data_risk.csv')

In [3]: credit.head()
Out[3]: Unnamed: 0  Age  Sex  Job  Housing  Saving accounts  Checking account \
          0      0  67  male  2      own              NaN              little
```

1	1	22	female	2	own	little	moderate
2	2	49	male	1	own	little	NaN
3	3	45	male	2	free	little	little
4	4	53	male	2	free	little	little

	Credit amount	Duration	Purpose	Risk
0	1169	6	radio/TV	good
1	5951	48	radio/TV	bad
2	2096	12	education	good
3	7882	42	furniture/equipment	good
4	4870	24	car	bad

```
In [4]: del credit['Unnamed: 0']
```

Dropping unnecessary column named Unnamed: 0

The summary statistics are given in the following code. Accordingly, the average age of the customers is roughly 35, average job type is skilled, average credit amount and duration are nearly 3,271 and 21, respectively. Additionally, the summary statistics tell us that the credit amount variable shows a relatively high standard deviation as expected. The duration and age variables have a very similar standard deviation, but the duration moves within a narrower interval as its minimum and maximum values are 4 and 72, respectively. As job is a discrete variable, it is natural to expect low dispersion and we have it:

```
In [5]: credit.describe()
Out[5]:
```

	Age	Job	Credit amount	Duration
count	1000.000000	1000.000000	1000.000000	1000.000000
mean	35.546000	1.904000	3271.258000	20.903000
std	11.375469	0.653614	2822.736876	12.058814
min	19.000000	0.000000	250.000000	4.000000
25%	27.000000	2.000000	1365.500000	12.000000
50%	33.000000	2.000000	2319.500000	18.000000
75%	42.000000	2.000000	3972.250000	24.000000
max	75.000000	3.000000	18424.000000	72.000000

In what follows, the distribution of the numerical variables in the dataset are examined via histogram and it turns out none of the variables follow a normal distribution.

The age, credit amount, and duration variables are positively skewed as we can see in Figure 6-1, generated by the following:

```
In [6]: import matplotlib.pyplot as plt
import seaborn as sns; sns.set()
plt.rcParams["figure.figsize"] = (10,6)

In [7]: numerical_credit = credit.select_dtypes(exclude='O')

In [8]: plt.figure(figsize=(10, 8))
k = 0
cols = numerical_credit.columns
for i, j in zip(range(len(cols)), cols):
    k +=1
    plt.subplot(2, 2, k)
    plt.hist(numerical_credit.iloc[:, i])
    plt.title(j)
```

Setting a fix figure size

Dropping the object type variables to obtain all numerical variables

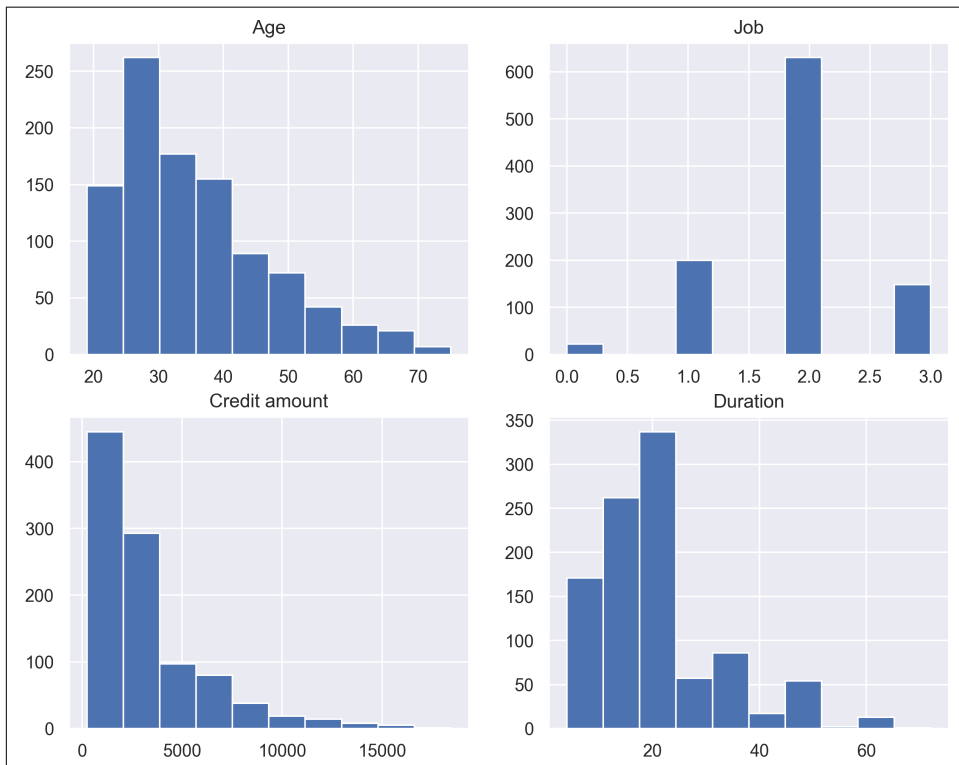


Figure 1. Credit risk data histogram

Figure 1 shows the distribution of age, job, credit amount, and duration variables. Aside from the job variable, which is a discrete variable, all other variables have skewed distributions.

The elbow method, as a first method, is introduced in the following code snippet and the resulting Figure 2. To find the optimal number of clusters, we observe the slope of the curve and decide the cut-off point at which the curve gets flatter—that is, the slope of the curve gets lower. As it gets flatter, the inertia, telling us how far away the points within a cluster are located, decreases, which is nice for the purpose of clustering. On the other hand, as we allow inertia to decrease, the number of clusters increases, which makes the analysis more complicated. Given that trade-off, the stopping criteria is the point where the curve gets flatter. In code:

```
In [9]: from sklearn.preprocessing import StandardScaler
        from sklearn.cluster import KMeans
        import numpy as np

In [10]: scaler = StandardScaler()
         scaled_credit = scaler.fit_transform(numerical_credit)

In [11]: distance = []
         for k in range(1, 10):
             kmeans = KMeans(n_clusters=k)
             kmeans.fit(scaled_credit)
             distance.append(kmeans.inertia_)

In [12]: plt.plot(range(1, 10), distance, 'bx-')
         plt.xlabel('k')
         plt.ylabel('Inertia')
         plt.title('The Elbow Method')
         plt.show()
```

Applying standardization for scaling purpose

Running K-means algorithm

Calculating inertia and storing into a list named distance

Figure 2 shows that the curve gets flatter after four clusters. Thus, the elbow method suggests that we stop at four clusters.

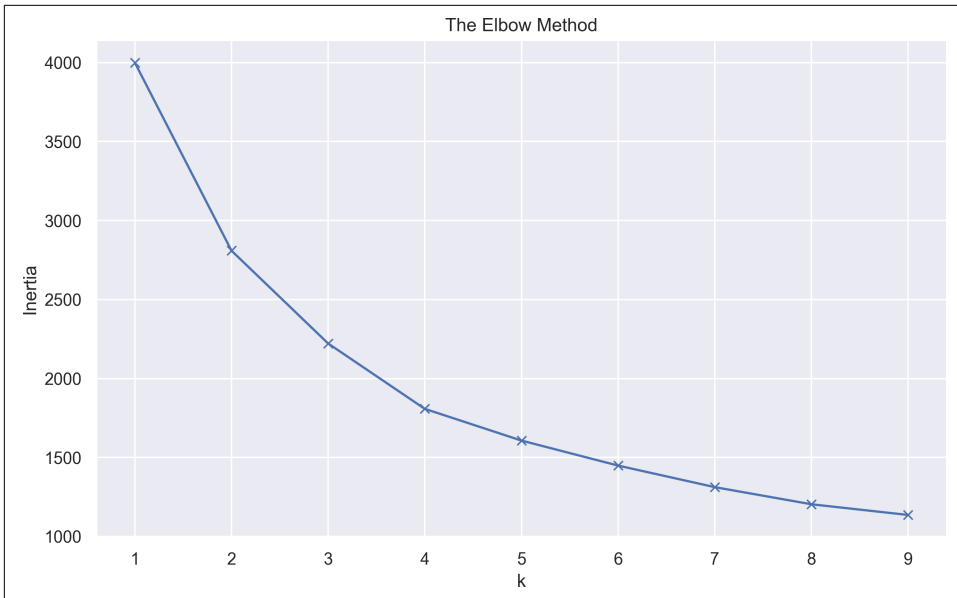


Figure 2. Elbow method

The following code, resulting in [Figure 3](#), presents Silhouette scores on the x-axis for clusters 2 to 10. Given the average Silhouette score represented by the dashed line, the optimal number of clusters can be two:

```
In [13]: from sklearn.metrics import silhouette_score
         from yellowbrick.cluster import SilhouetteVisualizer

In [14]: fig, ax = plt.subplots(4, 2, figsize=(25, 20))
         for i in range(2, 10):
             km = KMeans(n_clusters=i)
             q, r = divmod(i, 2)
             visualizer = SilhouetteVisualizer(km, colors='yellowbrick',
                                                ax=ax[q - 1][r])
             visualizer.fit(scaled_credit)
             ax[q - 1][r].set_title("For Cluster_" + str(i))
             ax[q - 1][r].set_xlabel("Silhouette Score")
```

Importing the `silhouette_score` module to calculate Silhouette score

Importing the `SilhouetteVisualizer` module to draw Silhouette plots

Using `divmod` for configuring labels, as it returns the quotient (q) and remainder (r)

Plotting the Silhouette scores

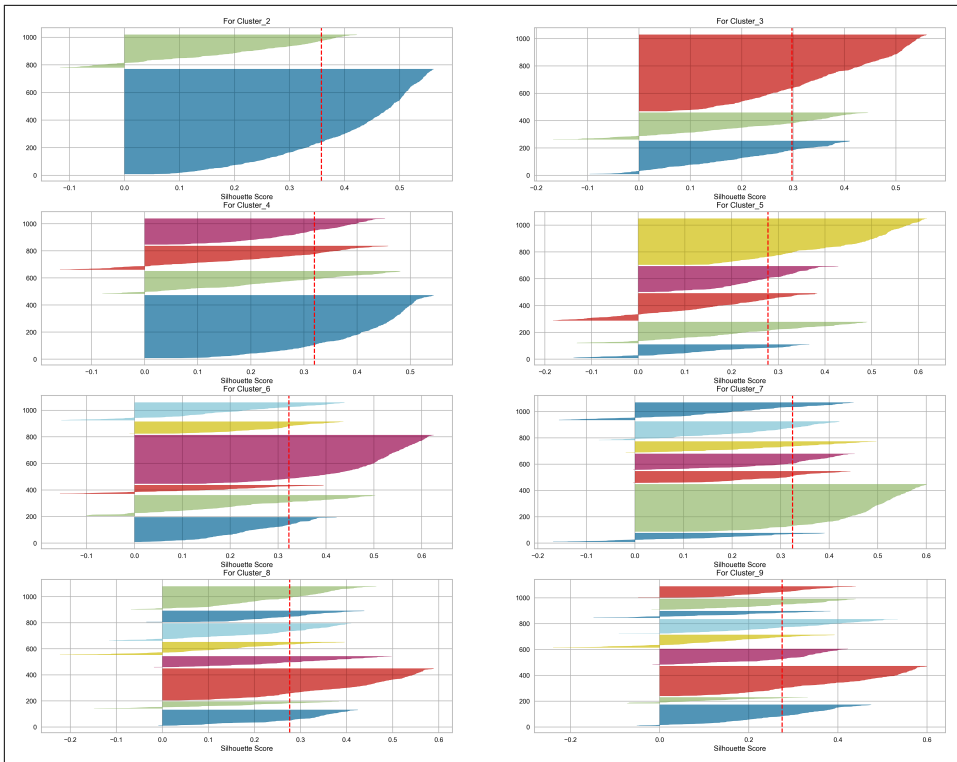


Figure 3. Silhouette score

As mentioned, the CH method is a convenient tool for finding optimal clustering, and the following code shows how we can use this method in Python, resulting in Figure 4. We are looking for the highest CH score, and we'll see that it is obtained at cluster 2:

```
In [15]: from yellowbrick.cluster import KElbowVisualizer
          model = KMeans()
          visualizer = KElbowVisualizer(model, k=(2, 10),
                                         metric='calinski_harabasz',
                                         timings=False)
          visualizer.fit(scaled_credit)
          visualizer.show()
Out[:]: <Figure size 576x396 with 0 Axes>
```

Importing KElbowVisualizer to draw the CH score

Visualizing the CH metric

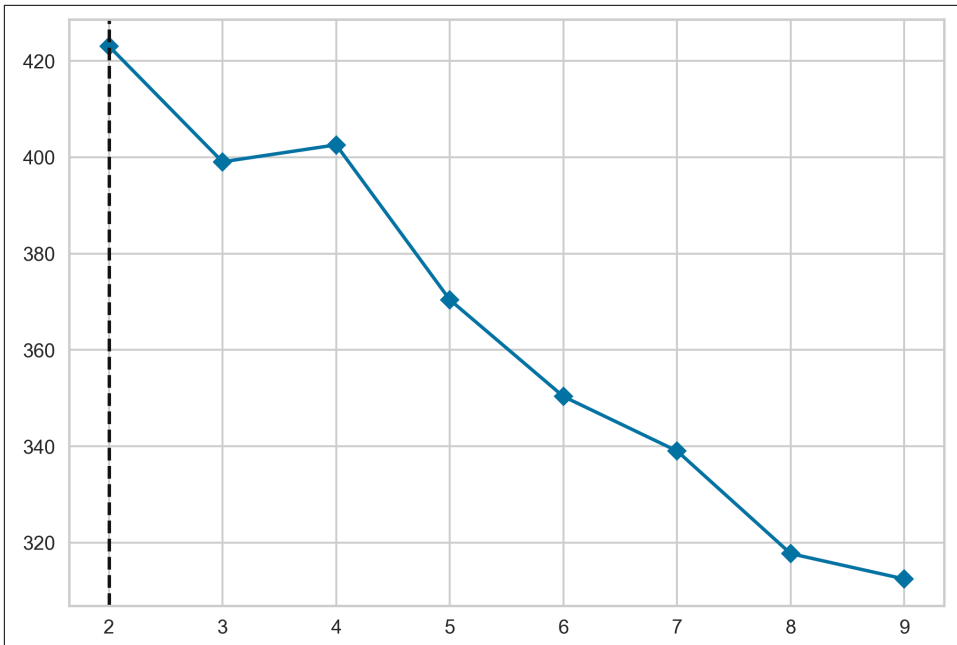


Figure 4. *We* CH method

Figure 4 shows that the elbow occurs at the second cluster, indicating that stopping at two clusters is the optimum decision.

The last step for finding the optimal number of clusters is gap analysis, resulting in Figure 5:

```
In [16]: from gap_statistic.optimalK import OptimalK

In [17]: optimalK = OptimalK(n_jobs=8, parallel_backend='joblib')
          n_clusters = optimalK(scaled_credit, cluster_array=np.arange(1, 10))

In [18]: gap_result = optimalK.gap_df
          gap_result.head()
Out[18]:
```

	n_clusters	gap_value	gap*	ref_dispersion_std	sk	\
0	1.0	0.889755	5738.286952	54.033596	0.006408	
1	2.0	0.968585	4599.736451	366.047394	0.056195	
2	3.0	1.003974	3851.032471	65.026259	0.012381	
3	4.0	1.044347	3555.819296	147.396138	0.031187	
4	5.0	1.116450	3305.617917	27.894622	0.006559	

	sk*	diff	diff*
0	6626.296782	-0.022635	6466.660374
1	5328.109873	-0.023008	5196.127130
2	4447.423150	-0.009186	4404.645656
3	4109.432481	-0.065543	4067.336067
4	3817.134689	0.141622	3729.880829

```
In [19]: plt.plot(gap_result.n_clusters, gap_result.gap_value)
          min_ylim, max_ylim = plt.ylim()
          plt.axhline(np.max(gap_result.gap_value), color='r',
                      linestyle='dashed', linewidth=2)
          plt.title('Gap Analysis')
          plt.xlabel('Number of Cluster')
          plt.ylabel('Gap Value')
          plt.show()
```

Importing the OptimalK module for calculating the gap statistic

Running gap statistic using parallelization

Identifying the number of clusters based on the gap statistic

Storing the result of gap analysis

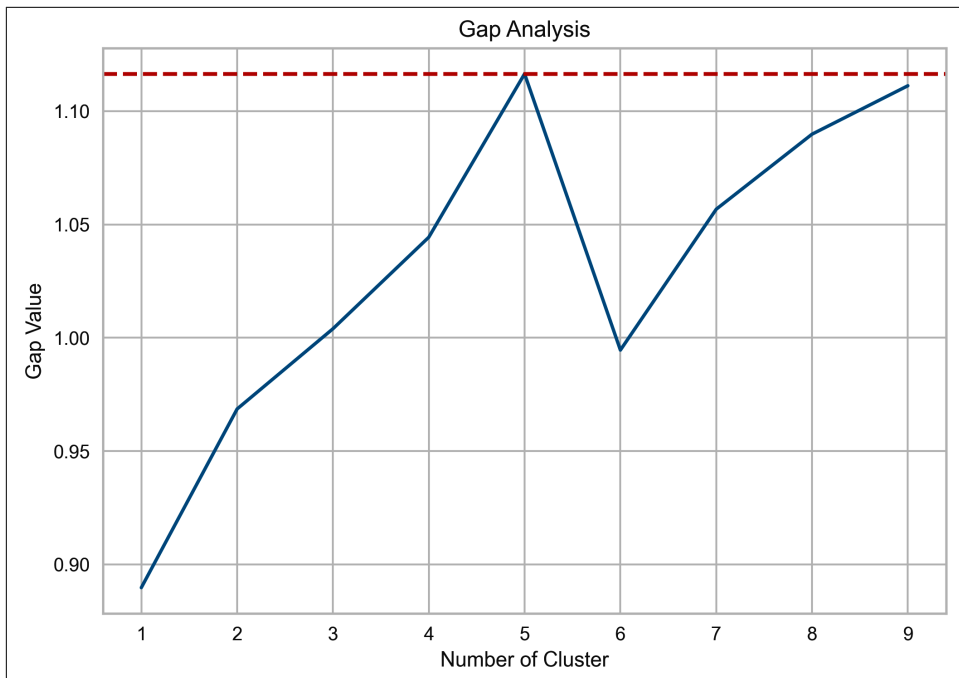


Figure 5. Gap analysis

What we observe in [Figure 5](#) is a sharp increase to the point at which the gap value reaches its peak, and the analysis suggests stopping at the maximum value at which we find the optimal number for clustering. In this case, we find the value at cluster 5, so this is the cut-off point.

In light of these discussions, two clusters are chosen to be the optimal number of clusters, and the K-means clustering analysis is conducted accordingly. To illustrate, given the clustering analysis, let us visualize 2-D clusters with the following, resulting in Figure 6:

```
In [20]: kmeans = KMeans(n_clusters=2)
         clusters = kmeans.fit_predict(scaled_credit)

In [21]: plt.figure(figsize=(10, 12))
         plt.subplot(311)
         plt.scatter(scaled_credit[:, 0], scaled_credit[:, 2],
                     c=kmeans.labels_, cmap='viridis')
         plt.scatter(kmeans.cluster_centers_[0, 0],
                     kmeans.cluster_centers_[0, 2], s = 80,
                     marker= 'x', color = 'k')
         plt.title('Age vs Credit')
         plt.subplot(312)
         plt.scatter(scaled_credit[:, 0], scaled_credit[:, 2],
                     c=kmeans.labels_, cmap='viridis')
         plt.scatter(kmeans.cluster_centers_[0, 0],
                     kmeans.cluster_centers_[0, 2], s = 80,
                     marker= 'x', color = 'k')
         plt.title('Credit vs Duration')
         plt.subplot(313)
         plt.scatter(scaled_credit[:, 2], scaled_credit[:, 3],
                     c=kmeans.labels_, cmap='viridis')
         plt.scatter(kmeans.cluster_centers_[0, 2],
                     kmeans.cluster_centers_[0, 3], s = 120,
                     marker= 'x', color = 'k')
         plt.title('Age vs Duration')
         plt.show()
```

Figure 6 presents the behavior of the observations and cross sign x indicates the cluster center, i.e., the centroid. Age represents the more dispersed data, and the centroid of the age variable is located above the credit variable. Two continuous variables, namely credit and duration, are shown in the second subplot of Figure 6-6, where we observe clearly separated clusters. This figure suggests that the duration variable is more volatile compared to the credit variable. In the last subplot, the relationship between age and duration is examined via scatter analysis. It turns out that there are many overlapping observations across these two variables.



Figure 6. K-means clusters

Probability of Default Estimation with Logistic Regression

Having obtained the clusters, we are able to treat customers with similar characteristics the same way—that is, the model learns in an easier and more stable way if data with similar distributions is provided. Conversely, using all the customers for the entire sample might result in poor and unstable predictions.

This section is ultimately about calculating the probability of default with Bayesian estimation, but let's first look at logistic regression for the sake of comparison.¹

Logistic regression is a classification algorithm, widely applicable in the finance industry. In other words, it proposes a regression approach to the classification problem. Logistic regression seeks to predict discrete output, taking into account some independent variables.

Let X be the set of independent variables and Y be a binary (or multinomial) output. Then, the conditional probability becomes:

$$\Pr(Y = 1 | X = x)$$

This can be read as: given the values of X , what is the probability of having Y as 1? As the dependent variable of logistic regression is of the probabilistic type, we need to make sure the dependent variable cannot take on values other than between 0 and 1.

To this aim, a modification is applied known as *logistic (logit) transformation*, which is simply the log of the odds ratio ($p / 1 - p$):

$$\log\left(\frac{p}{1-p}\right)$$

And the logistic regression model takes the following form:

$$\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 x$$

Solving p results in:

$$p = \frac{e^{\beta_0 + \beta_1 x}}{1 + e^{\beta_0 + \beta_1 x}}$$

Let's start off our application by preparing the data. First, we distinguish the clusters as 0 and 1. The credit data has a column named `risk`, suggesting the risk level of the customers. Next, the number of observations per risk in cluster 0 and cluster 1 are examined; it turns out we have 571 and 129 good customers in the cluster 0 and 1, respectively. In code:

¹ It is useful to run logistic regression to initialize results for priors in Bayesian estimation.

```

In [22]: clusters, counts = np.unique(kmeans.labels_, return_counts=True)

In [23]: cluster_dict = {}
         for i in range(len(clusters)):
             cluster_dict[i] = scaled_credit[np.where(kmeans.labels_==i)]

In [24]: credit['clusters'] = pd.DataFrame(kmeans.labels_)

In [25]: df_scaled = pd.DataFrame(scaled_credit)
         df_scaled['clusters'] = credit['clusters']

In [26]: df_scaled['Risk'] = credit['Risk']
         df_scaled.columns = ['Age', 'Job', 'Credit amount',
                             'Duration', 'Clusters', 'Risk']

In [27]: df_scaled[df_scaled.Clusters == 0]['Risk'].value_counts()
Out[27]: good      571
         bad       193
         Name: Risk, dtype: int64

In [28]: df_scaled[df_scaled.Clusters == 1]['Risk'].value_counts() ⑤
Out[28]: good      129
         bad       107
         Name: Risk, dtype: int64

```

Obtaining cluster numbers

Based on the cluster numbers, differentiating the clusters and storing them in a dictionary called `cluster_dict`

Creating a `clusters` column using K-means labels

Observing the number of observations of categories within a cluster

⑤ Finding number of observations per category

Next, we draw a couple of bar plots to show the difference of the number of observations per risk level category (Figures 7 and 8):

```

In [29]: df_scaled[df_scaled.Clusters == 0]['Risk'].value_counts()\
         .plot(kind='bar',
         figsize=(10, 6),
         title="Frequency of Risk Level");

In [30]: df_scaled[df_scaled.Clusters == 1]['Risk'].value_counts()\
         .plot(kind='bar',
         figsize=(10, 6),
         title="Frequency of Risk Level");

```

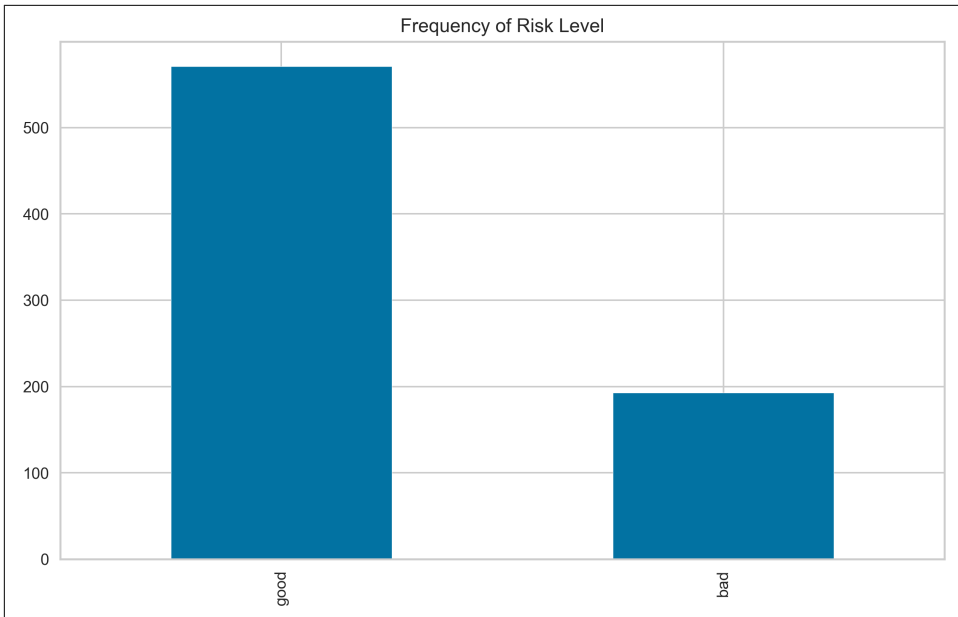


Figure 7. Frequency of risk level of the first cluster

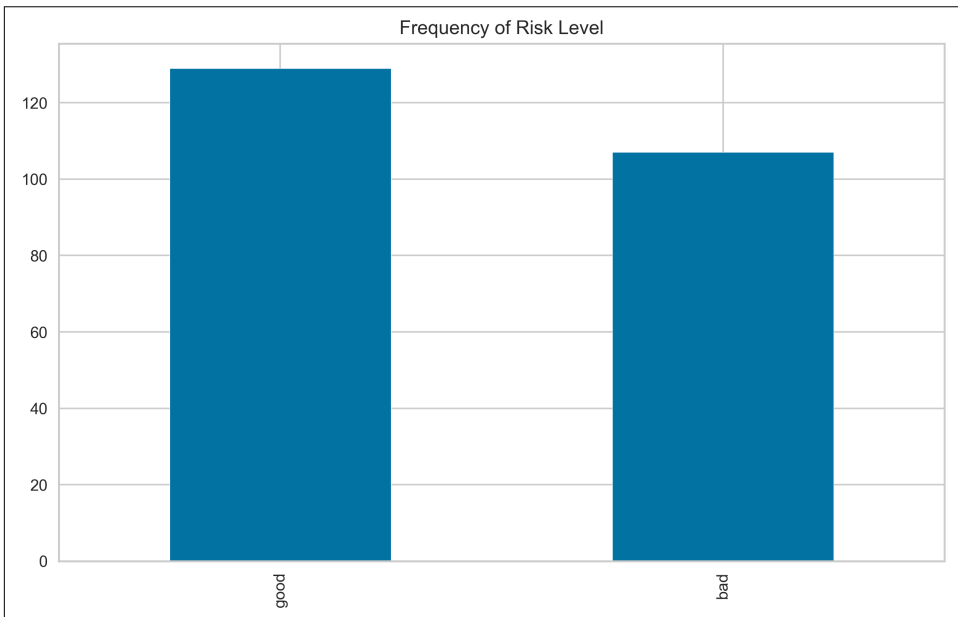


Figure 8. Frequency of risk level of the second cluster

Based on the clusters we defined previously, we can analyze the frequency of risk level by histogram. [Figure 7](#) shows that there is an imbalance distribution across risk level in the first cluster, whereas the frequency of good and bad risk levels are more balanced, if not perfectly balanced, in [Figure 8](#).

At this point, let's take a step back and focus on an entirely different problem: *class imbalance*. In credit risk analysis, it is not uncommon to have a class imbalance problem. Class imbalance arises when one class dominates over another. To illustrate, in our case, given the data obtained from the first cluster, we have 571 customers with a good credit record and 193 customers with a bad one. As can be readily observed, customers with good credit records dominate over customers with bad records; that is basically what we refer to as a class imbalance.

There are numerous ways to handle this issue: up-sampling, down-sampling, the synthetic minority oversampling technique (SMOTE), and the edited nearest neighbor (ENN) rule. To take advantage of a hybrid approach, we'll incorporate a combination of SMOTE and ENN so we can clean the unwanted overlapping observations between classes, which will help us detect the optimal balancing ratio and, in turn, boost the predictive performance (Tuong et al. 2018). Converting imbalanced data into balanced data will be our first step in predicting the probability of default, but please note that we will merely apply this technique to the data obtained from the first cluster.

Now, we next apply a train-test split. To do that, we need to convert the categorical variable `Risk` into a discrete variable. The category `good` takes a value of 1, and `bad` takes a value of 0. In a train-test split, 80% of the data is devoted to training samples and 20% of is allocated to the test sample:

```
In [31]: from sklearn.model_selection import train_test_split

In [32]: df_scaled['Risk'] = df_scaled['Risk'].replace({'good': 1, 'bad': 0})

In [33]: X = df_scaled.drop('Risk', axis=1)
         y = df_scaled.loc[:, ['Risk', 'Clusters']]

In [34]: X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                             test_size=0.2,
                                                             random_state=42)

In [35]: first_cluster_train = X_train[X_train.Clusters == 0].iloc[:, :-1]
         second_cluster_train = X_train[X_train.Clusters == 1].iloc[:, :-1]
```

Discretization of the variable

Creating data based on the first cluster and dropping last column from X_train

Creating data based on the second cluster and dropping last column from X_train

After these preparations, we are ready to move ahead and run the logistic regression to predict the probability of default. The library that we'll make use of is called statsmodels, and it is allowed to have a summary table. The following result is based on the first cluster data. According to the result, the age, credit amount, and job variables are positively related with the creditworthiness of customer, while a negative association emerges between the dependent and duration variables. This finding suggests that all the estimated coefficients reveal statistically significant results at a 1% significance level. A general interpretation would be that a slide in duration and a surge in credit amount, age, and job imply a high probability of default:

```
In [36]: import statsmodels.api as sm
         from sklearn.linear_model import LogisticRegression
         from sklearn.metrics import roc_auc_score, roc_curve
         from imblearn.combine import SMOTEENN
         import warnings
         warnings.filterwarnings('ignore')
```

```
In [37]: X_train1 = first_cluster_train
         y_train1 = y_train[y_train.Clusters == 0]['Risk']
         smote = SMOTEENN(random_state = 2)
         X_train1, y_train1 = smote.fit_resample(X_train1, y_train1.ravel())
         logit = sm.Logit(y_train1, X_train1) 5
         logit_fit1 = logit.fit() 6
         print(logit_fit1.summary())
         Optimization terminated successfully.
         Current function value: 0.479511
         Iterations 6
```

Logit Regression Results

=====						
Dep. Variable:	y	No. Observations:	370			
Model:	Logit	Df Residuals:	366			
Method:	MLE	Df Model:	3			
Date:	Wed, 01 Dec 2021	Pseudo R-squ.:	0.2989			
Time:	20:34:31	Log-Likelihood:	-177.42			
converged:	True	LL-Null:	-253.08			
Covariance Type:	nonrobust	LLR p-value:	1.372e-32			
=====						
	coef	std err	z	P> z	[0.025	0.975]

Age	1.3677	0.164	8.348	0.000	1.047	1.689
Job	0.4393	0.153	2.873	0.004	0.140	0.739

Credit amount	1.3290	0.305	4.358	0.000	0.731	1.927
Duration	-1.2709	0.246	-5.164	0.000	-1.753	-0.789

=====

Importing SMOTEENN to deal with the class imbalance problem

Creating y_train based on cluster 0 and risk level

Running the SMOTEENN method with a random state of 2

Turning the imbalanced data into balanced data

⑤ Configuring the logistic regression model

⑥ Running the logistic regression model

In what follows, prediction analysis is conducted by creating different datasets based on clusters. For the sake of testing, the following analysis is done with test data, and results in **Figure 9**:

```
In [38]: first_cluster_test = X_test[X_test.Clusters == 0].iloc[:, :-1]
         second_cluster_test = X_test[X_test.Clusters == 1].iloc[:, :-1]

In [39]: X_test1 = first_cluster_test
         y_test1 = y_test[y_test.Clusters == 0]['Risk']
         pred_prob1 = logit_fit1.predict(X_test1)

In [40]: false_pos, true_pos, _ = roc_curve(y_test1.values, pred_prob1)
         auc = roc_auc_score(y_test1, pred_prob1) ⑤
         plt.plot(false_pos, true_pos, label="AUC for cluster 1={:.4f} "
                  .format(auc))
         plt.plot([0, 1], [0, 1], linestyle = '--', label='45 degree line')
         plt.legend(loc='best')
         plt.title('ROC-AUC Curve 1')
         plt.show()
```

Creating first test data based on cluster 0

Creating second test data based on cluster 1

Running prediction using X_test1

Obtaining false and true positives using roc_curve function

⑤ Compute the roc-auc score

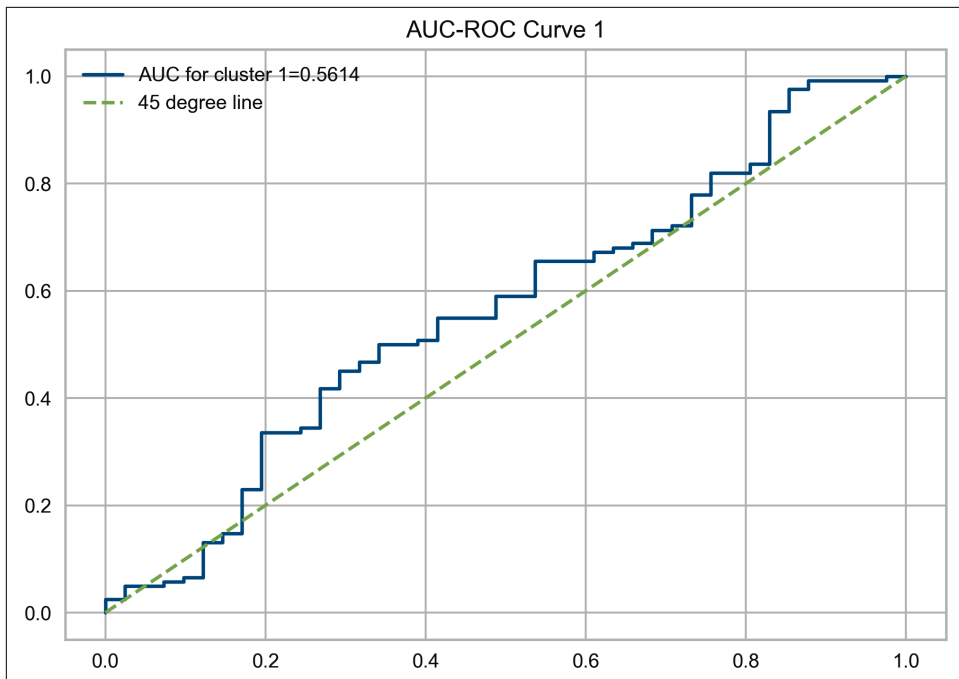


Figure 9. ROC-AUC curve of the first cluster

The ROC-AUC curve is a convenient tool in the presence of imbalanced data. The ROC-AUC curve in [Figure 6-9](#) suggests that the performance of the model is not very good, because it moves just above the 45-degree line. Generally speaking, given the test results, a good ROC-AUC curve should be close to 1, implying that there is a close-to-perfect separation.

Moving on to the second set of training samples obtained from the second cluster, the signs of the estimated coefficients of job, duration, and age are positive, suggesting that customers with job type of 1 and having larger duration tend to default, and the credit amount variable shows a negative relation with dependent variable. However, all the estimated coefficients are statistically insignificant at 95% confidence interval; therefore, it makes no sense to further interpret the findings.

Similar to what we did with the first set of test data, we create a second set of test data to run the prediction to draw the ROC-AUC curve, resulting in [Figure 10](#):

```
In [41]: X_train2 = second_cluster_train
         y_train2 = y_train[y_train.Clusters == 1]['Risk']
         logit = sm.Logit(y_train2, X_train2)
         logit_fit2 = logit.fit()
         print(logit_fit2.summary())
Optimization terminated successfully.
Current function value: 0.688152
```

Iterations 4

Logit Regression Results

Dep. Variable:	Risk	No. Observations:	199
Model:	Logit	Df Residuals:	195
Method:	MLE	Df Model:	3
Date:	Wed, 01 Dec 2021	Pseudo R-squ.:	-0.0008478
Time:	20:34:33	Log-Likelihood:	-136.94
converged:	True	LL-Null:	-136.83
Covariance Type:	nonrobust	LLR p-value:	1.000

	coef	std err	z	P> z	[0.025	0.975]
Age	0.0281	0.146	0.192	0.848	-0.259	0.315
Job	0.1536	0.151	1.020	0.308	-0.142	0.449
Credit amount	-0.1090	0.115	-0.945	0.345	-0.335	0.117
Duration	0.1046	0.126	0.833	0.405	-0.142	0.351

```
In [42]: X_test2 = second_cluster_test
         y_test2 = y_test[y_test.Clusters == 1]['Risk']
         pred_prob2 = logit_fit2.predict(X_test2)

In [43]: false_pos, true_pos, _ = roc_curve(y_test2.values, pred_prob2)
         auc = roc_auc_score(y_test2, pred_prob2)
         plt.plot(false_pos, true_pos, label="AUC for cluster 2={:.4f} "
                  .format(auc))
         plt.plot([0, 1], [0, 1], linestyle = '--', label='45 degree line')
         plt.legend(loc='best')
         plt.title('ROC-AUC Curve 2')
         plt.show()
```

Given the test data, the result shown in [Figure 6-10](#) is worse than the previous application, as can be confirmed by the AUC score of 0.4064. Considering this data, we are far from saying that logistic regression is doing a good job of modeling probability of default using the German credit risk dataset.

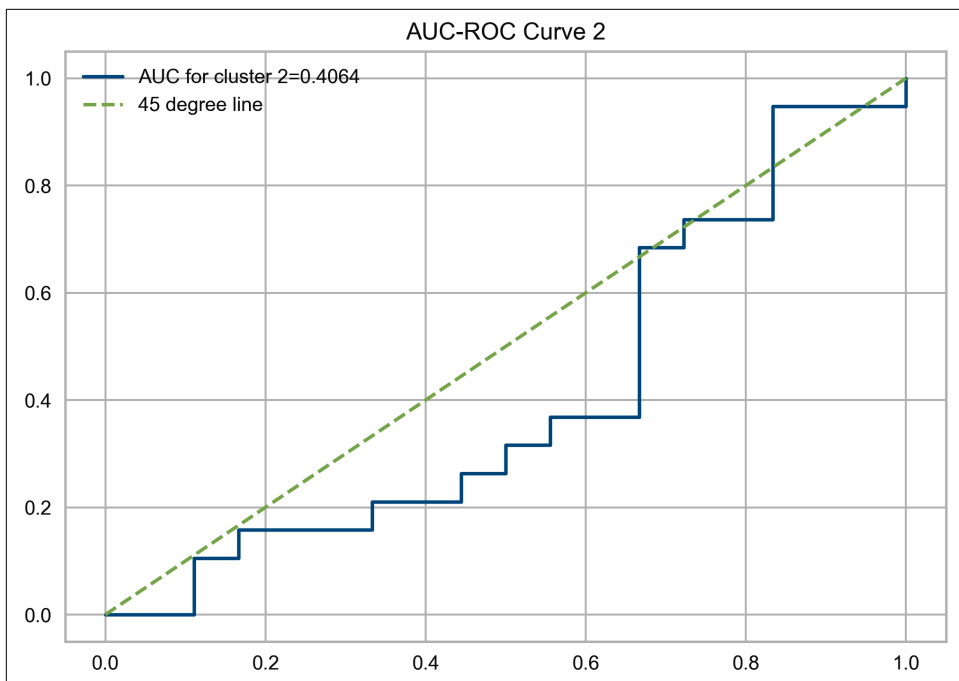


Figure 10. ROC-AUC curve of the second cluster